# Explainability in Reinforcement Learning

Maxime Alaarabiou

Wednesday 2nd September, 2026

Compiled on 2/9/2026 at 11:58

# Chapter 1

# Deep Learning

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Network (NN). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this manuscript. Its structure is as follows:

Section 1.1: An overview of the historical developments that led to the emergence of the field.

Section 1.2: A formalization of the objective underlying DL algorithms.

Section 1.3: An examination of the structure of artificial NN.

Section 1.4: An explanation of how the learning process shapes NN.

Section 1.5: A discussion of how such models are evaluated.

Section 1.6: A summary of the concepts introduced in the chapter and a discussion of the theoretical limitations of DL.

## 1.1 Historical Developments

NN and DL emerged from an ambition to study the expressive capabilities of organic brain. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [McCulloch and Pitts, 1943]. This work established the key idea that networks of simple units can give rise to complex computations. The focus was on representational power, not on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [Rosenblatt, 1958]. It was designed to adjust its parameters from input–output pairs in order to approximate a target function. The perceptron fundamental limitations were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or [Minsky and Papert, 1969]. At the same time, it was demonstrated that stacking multiple perceptrons could, in principle, represent such functions. However, no effective training procedure

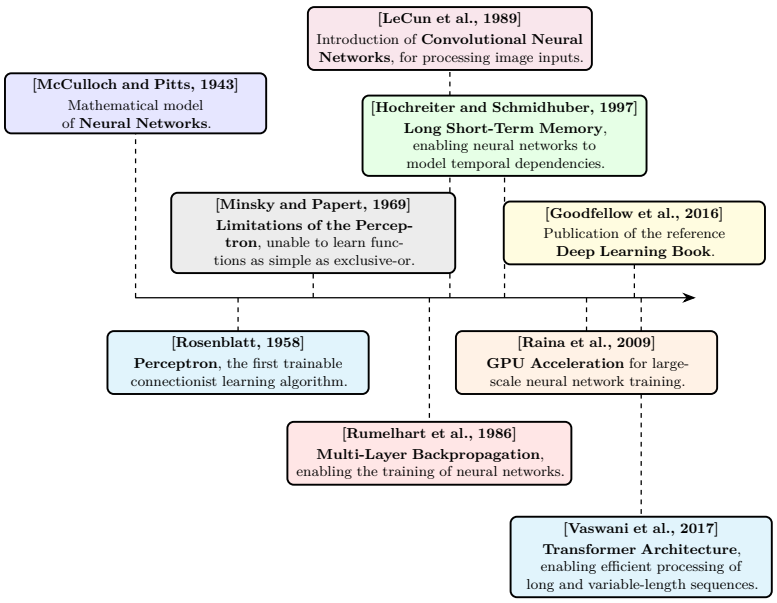


Figure 1.1: Evolution of deep learning.

for these multi-layer architectures was available at the time. It was only in 1986 that researchers introduced gradient descent as an effective method for training deep NN [Rumelhart et al., 1986]. From this point onward, the emergence of DL is retrospectively situated, even though the term itself appeared later. DL is defined as the training of deep NN using gradient-based optimization methods.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NN can represent a wide class of functions [Hornik et al., 1989], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [Hilbert and López, 2011]. The use of graphics processing units for large-scale training, demonstrated in 2009, greatly increased computational capacity and made it feasible to train large-scale models [Raina et al., 2009, Krizhevsky et al., 2012].

At the same time, researchers developed architectures specifically designed for different types of data, allowing DL to address an increasingly wide range of problems. Convolutional Neural Network (CNN) were introduced in 1989 to process images, using local, weight-shared filters that exploit the spatial structure of pixel grids [LeCun et al., 1989]. Long Short-Term Memory (LSTM) networks were later proposed in 1997 to process temporal sequences, introducing

gating mechanisms able to retain relevant information over long time horizons [Hochreiter and Schmidhuber, 1997]. More recently, the Transformer architecture, relying entirely on the attention mechanism, was introduced in 2017 to process long, variable-length sequences of strongly interconnected elements, allowing every element to attend directly to every other element regardless of their distance within the sequence [Vaswani et al., 2017]. This growing body of knowledge was consolidated in 2016 in a reference textbook that formalized DL as a coherent field of study [Goodfellow et al., 2016].

These changes paved the way for the rise of DL as it is known today. DL has achieved remarkable success across a wide range of domains, including computer vision [Krizhevsky et al., 2012], complex games [Silver et al., 2017], content recommendation [Covington et al., 2016], bio-chemistry [Jumper et al., 2021], as well as the now essential digital assistants based on Large Language Model (LLM) [Brown et al., 2020]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

Figure 1.1 provides a timeline of the key milestones that shaped deep learning.

## 1.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are progressively updated during training so as to make it a better approximation of the target function. It gradually becomes capable of capturing the relationship between inputs and outputs through successive parameter updates during training. For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> "A computer program is said to learn from experience $E$ with respect to some class of tasks $T$ and performance measure $P$, if its performance at tasks in $T$, as measured by $P$, improves with experience $E$."
> [Mitchell, 1997]

When this formulation is adapted to DL, the following correspondences hold:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task $T$ consists of learning a NN that provides a good approximation of the target function.
- The performance measure $P$ corresponds to a function used to evaluate the quality of this approximation.
- The experience $E$ takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

The function to be approximated is denoted by $f$. As with any function, it defines a mapping from an input to an output: $x \in \mathcal{X}$ denotes an input and $y \in \mathcal{Y}$ the corresponding output. The function $f$ is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces throughout this manuscript. This assumption is adopted solely to facilitate the presentation, as the framework can be readily extended to more general settings. The notation $\hat{\cdot}$ indicates that a given object is an approximation of another. Since the goal of the trained NN is to approximate $f$, this approximation is denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $x \in \mathcal{X}$, the output of the approximating function $\hat{f}(x)$ is denoted by $\hat{y}$.

It is important to emphasize that the target function $f$ is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, only a finite set of observations generated by $f$ is available, organized in what is called a dataset. To distinguish between the different samples, an index $i$ is introduced, allowing input–output pairs of the form $(x^{(i)}, y^{(i)})$ to be defined. This leads to the following formulation for a dataset $\mathcal{D}$ of size $N$, where $N$ denotes the number of available examples:

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}. \tag{1.1}$$

Now that the representation of the target function $f$ has been established, the next question is what makes a function $\hat{f}$ a good approximation of it. For a given input $x^{(i)}$, this means that the NN output $\hat{y}^{(i)}$ should be close to the true output $y^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is introduced that measures the discrepancy between predictions and target values. This function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the Mean Squared Error (MSE), defined as:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2. \tag{1.2}$$

Thanks to this loss, it is now possible to rigorously define the performance measure $P$. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(x^{(i)}), y^{(i)}). \tag{1.3}$$

This formulation captures the mathematical core of the learning objective, namely finding a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$.

Since the target function $f$ may take many different forms, $\hat{f}$ must be expressive enough to approximate a broad range of such functions. NN architectures are well suited to this requirement, as they can represent a wide range of functional mappings, whose composition is detailed in the following section.

## 1.3 Neural Network Structure

NN are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation, the focus is placed on the MultiLayer Perceptron (MLP) architecture. Although more specialized architectures exist, the MLP forms a fundamental building block of DL. All the essential mechanisms are present in it, making it a particularly suitable framework for introducing key concepts.

This section therefore begins by examining the artificial neuron, the fundamental computational unit underlying the MLP. It then explains how these neurons are organized into layers. Finally, it concludes with a general description of the NN as a whole.

### 1.3.1 Neurons

The artificial neuron is the basic building block of NN. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(a_1, \ldots, a_n)$, its activation $a$ is given by:

$$a = \sigma\left(\sum_{i=1}^{n} w_i a_i + b\right), \tag{1.4}$$

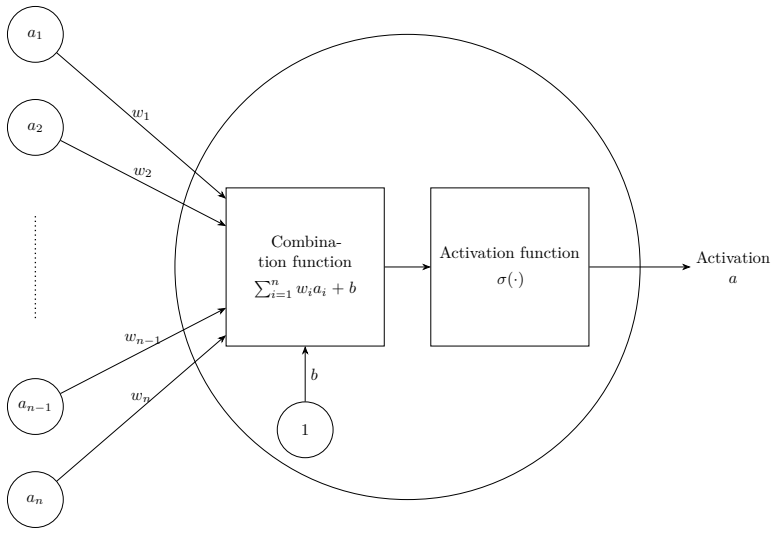


Figure 1.2: Artificiale neurone.

where:

- $w_i$ is the weight associated with the $i$-th incoming connection,
- $b$ is the bias,
- $\sigma$ is the activation function.

Figure 1.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias $b$ constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight $w$, relative to the others, indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function $\sigma$ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

### 1.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 1.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer

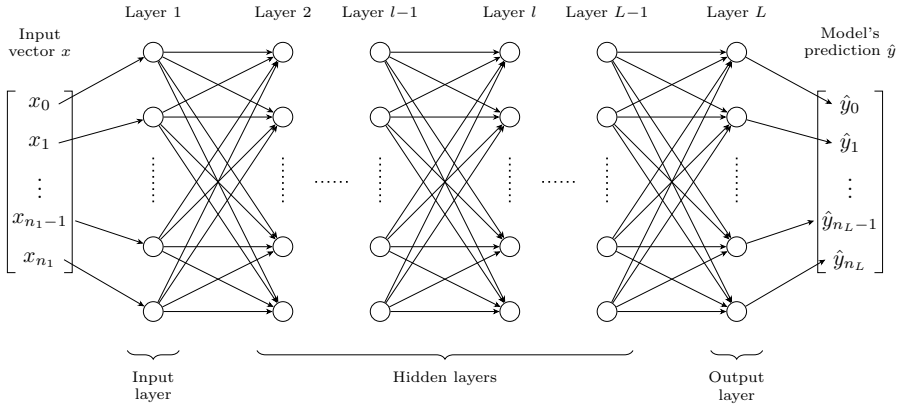


Figure 1.3: Multilayer Perceptron Architecture.

and sends its output to all neurons in the following layer. As a result, neurons within the same layer do not interact with each other.

Three types of layers are distinguished:

- an input layer, which directly receives input,
- one or more hidden layers (also referred to as intermediate layers), which perform intermediate transformations,
- an output layer, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer, as its neurons directly encode the components of the input vector $x$. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{y}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NN, especially in MLP, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [Krizhevsky et al., 2012]. Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer $l$ composed of $n_l$ neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [a_1^{(l)}, \ldots, a_{n_l}^{(l)}]. \tag{1.5}$$

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)} \left( \mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \tag{1.6}$$

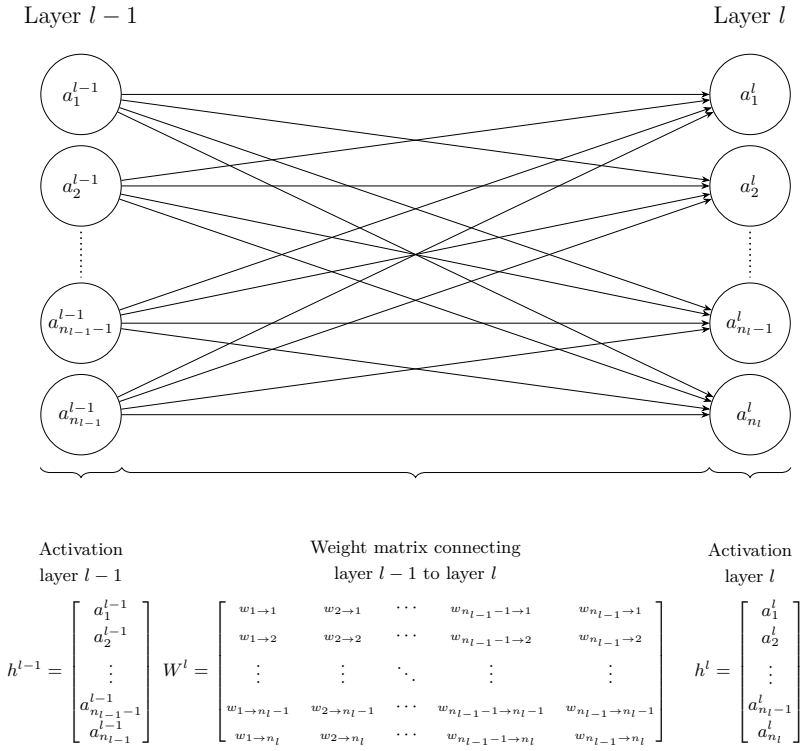


Figure 1.4: Interaction between layers.

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer $l$,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer $l$,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer $l$.

This matrix-vector representation is visualised in Figure 1.4, which highlights how each neuron in layer $l$ receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions, and in practice, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NN [Telgarsky, 2016]. Each layer can be viewed as an additional stage in the computation performed by the network. This suggests that deeper networks can represent increasingly complex functions by composing a larger number of transformations. The precise relationship between depth and function complexity remains primarily supported by empirical observations rather than formal theoretical justification [Goodfellow et al., 2016].

### 1.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 1) to the output layer (indexed $L$):

$$\hat{f}(x) = \mathbf{h}^{(L)} = \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L)}\right) = \hat{y}. \tag{1.7}$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} x & \text{if } l = 1, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) & \text{if } 2 \le l \le L. \end{cases} \tag{1.8}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\theta = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}. \tag{1.9}$$

In this manuscript, $\theta$ is a vector in $\mathbb{R}^P$, where $P$ denotes the total number of parameters, such that $\theta = [w_1, \dots, w_P]$. The notation $\hat{f}_\theta$ expresses the dependence of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, can enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to $\theta$, which controls the strength of the connections between neurons. The following sections examine how these parameters can be adjusted to obtain a better approximation of the target function.

## 1.4 Learning

Now that the structure of a NN has been detailed, the next step is to examine how its parameters are adjusted to accomplish the assigned task. This brings the discussion to the core of DL, namely the learning process itself. As a reminder, the objective of training a NN is to adjust its parameters so that the network provides a good approximation of the target function. Mathematically, this objective is expressed as:

$$\min_\theta \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.10}$$

As shown in Equation 1.10, the loss for example $i$ depends on the input-target pair $(x^{(i)}, y^{(i)})$, the loss function $\mathcal{L}$, the network architecture $\hat{f}$, and the parameters $\theta$. Since the dataset, loss function, and network architecture are fixed

during training, only the parameters $\theta$ are optimized. To simplify the notation, only the dependence on $\theta$ is kept, and the loss for example $i$ is written as

$$\mathcal{L}^{(i)}(\theta) = \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.11}$$

Now that the learning objective has been defined, the next question is how to modify the parameters $\theta$ to minimize the loss. The NN architecture $\hat{f}$ used to approximate the target function $f$ is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, the optimum cannot be determined analytically by directly studying the properties of the function.

Instead, approximate optimization methods are required to find suitable parameter values for NN. The class of algorithms used for this optimization is known as gradient descent methods. Numerous variants of gradient descent have been developed for NN training, each designed to improve specific aspects of the optimization process. For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

This section presents the learning process in three steps. The first derives the gradient of the loss using the chain rule. The second shows how this gradient is used to update the network parameters. The third discusses the convergence properties of the resulting algorithm.

### 1.4.1 Computing the Gradient

The first step of SGD is to randomly initialize the parameter vector $\theta \in \mathbb{R}^P$ for a given network architecture $\hat{f}$. This initial state is denoted by $\theta^{(1)}$. The goal is to iteratively update this vector so that the network gets better at approximating the target function. To do so, it is necessary to quantify the impact of each parameter $w_k$ on the loss $\mathcal{L}^{(i)}(\theta)$. This is done using the partial derivative of the loss with respect to each parameter, denoted by $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$.

Computing this derivative is not straightforward because a NN is composed of a sequence of functions, and the parameter $w_k$ can belong to any layer of the network. Its influence on the loss must therefore be propagated through all subsequent layers. To describe these dependencies, each layer can be viewed as a function, denoted by $\phi^{(l)}$ for layer $l$. This function takes the activation vector $\mathbf{h}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ produced by the previous layer as input. It produces the activation vector $\mathbf{h}^{(l)} \in \mathbb{R}^{n_l}$ of the current layer. This can be expressed as

$$\phi^{(l)} : \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}, \qquad \mathbf{h}^{(l)} = \phi^{(l)}(\mathbf{h}^{(l-1)}). \tag{1.12}$$

Because each layer function has several outputs that jointly depend on several inputs, the ordinary derivative is no longer sufficient to describe how it varies, and the Jacobian matrix is used instead as its generalization. The Jacobian matrix quantifies how a small change in each input component affects each output component. Consider layer $l$'s function, its Jacobian $J_{\phi^{(l)}}(\mathbf{h}^{(l-1)})$, evaluated at $\mathbf{h}^{(l-1)}$, collects the partial derivatives of every output component with respect

to every input component:

$$J_{\phi^{(l)}}(\mathbf{h}^{(l-1)}) = \begin{bmatrix} \frac{\partial h_1^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_1^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{n_l}^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_{n_l}^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \end{bmatrix}. \tag{1.13}$$

Consider two consecutive layers, $\phi^{(l)}$ and $\phi^{(l+1)}$. The chain rule states that the Jacobian of this composition is given by the product of the Jacobians of the two layers. Each Jacobian is evaluated at the input received by the corresponding layer:

$$J_{\phi^{(l+1)} \circ \phi^{(l)}}\left(\mathbf{h}^{(l-1)}\right) = J_{\phi^{(l+1)}}\left(\mathbf{h}^{(l)}\right) J_{\phi^{(l)}}\left(\mathbf{h}^{(l-1)}\right). \tag{1.14}$$

The network is composed of a sequence of layer functions $\phi^{(1)}, \ldots, \phi^{(L)}$. Each layer depends only on the output of the previous layer. Consider a parameter $w_k$ belonging to layer $l$. A change in $w_k$ first affects the output $\mathbf{h}^{(l)}$ of its own layer. This change then propagates through the subsequent layers, affecting $\mathbf{h}^{(l+1)}$, and so on. The effect reaches the output $\mathbf{h}^{(L)}$, from which the loss $\mathcal{L}^{(i)}(\theta)$ is computed. This layer-by-layer propagation is referred to as backpropagation:

$$\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k} = J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})\, J_{\phi^{(L)}}(\mathbf{h}^{(L-1)}) \cdots J_{\phi^{(l+1)}}(\mathbf{h}^{(l)})\, J_{\phi^{(l)}}(w_k), \tag{1.15}$$

where:

- $J_{\mathcal{L}}(\mathbf{h}^{(L)}, y^{(i)})$ denotes the Jacobian of $\mathcal{L}$ with respect to its first argument, evaluated at $(\mathbf{h}^{(L)}, y^{(i)})$,
- each subsequent $J_{\phi^{(j+1)}}(\mathbf{h}^{(j)})$ denotes the Jacobian of the function $\phi^{(j+1)}$ with respect to its input $\mathbf{h}^{(j)}$,
- $J_{\phi^{(l)}}(w_k)$ denotes the Jacobian of the function $\phi^{(l)}$ with respect to the parameter $w_k$.

Multiplying these local Jacobians together therefore propagates the effect of $w_k$, layer by layer, all the way to the loss.

Once $\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_k}$ can be computed for every parameter, all of these values are stacked into one vector, called the gradient:

$$\nabla_\theta \mathcal{L}^{(i)}(\theta) = \left[\frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_1}, \ldots, \frac{\partial \mathcal{L}^{(i)}(\theta)}{\partial w_P}\right]. \tag{1.16}$$

Each component answers the same question for a different parameter, indicating whether a small increase in that parameter increases or decreases the loss, and by how much. A positive value means increasing the parameter increases the loss, whereas a negative value means the opposite. The gradient thus indicates, for every parameter, which direction to move in order to reduce the loss.

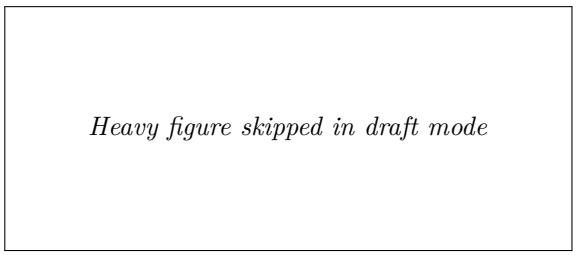


Figure 1.5: *Figure skipped in draft mode (set* `\draftfiguresfalse` *to render it).*

### 1.4.2 Update rule

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads to the gradient descent update rule:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_\theta \mathcal{L}^{(i)} \left(\theta^{(n)}\right), \tag{1.17}$$

where $\eta > 0$ is the learning rate, which controls the step size, and $\nabla_\theta \mathcal{L}^{(i)} \left(\theta^{(n)}\right)$ denotes the gradient of the loss with respect to the parameters, evaluated at the current values $\theta^{(n)}$.

This parameter $\eta$ must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\theta^{(1)}, \theta^{(2)}, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

Algorithm 1 summarizes this whole training procedure.

To develop an intuition for this update rule, Figure 1.5 illustrates it applied to the simple convex loss function $\mathcal{L}(w_1, w_2) = w_1^2 + w_2^2$, where the red path shows the parameter updates converging to the minimum. The point $\theta^{(1)}$ represents the initial parameter values, with $w_1$ and $w_2$ randomly initialized, and the subsequent points, from $\theta^{(2)}$ to $\theta^{(6)}$, represent the successive updates produced by applying Equation 1.17, progressively moving the trajectory toward the minimum of the loss function.

### 1.4.3 Convergence Properties

Having established how SGD updates the parameters, a natural question is whether this procedure is guaranteed to converge. It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, and no such guarantee holds. Figure 1.6 illustrates the effect of non-convex functions, different starting points lead to distinct local minima.

**Algorithm 1:** Stochastic Gradient Descent

**Input:**
Dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$
Architecture $\hat{f}$
Loss function $\mathcal{L}$
Learning rate $\eta > 0$
Initial parameters $\theta$ (optional)
**Output:**
Trained parameters $\theta$

---

**if** $\theta$ *is given* **then**
  $\theta^{(1)} \leftarrow \theta$;
**else**
  Initialize $\theta^{(1)}$ randomly;
$n \leftarrow 1$;
**while** *stopping criterion not satisfied* **do**
  Select a random example $(x^{(i)}, y^{(i)}) \in \mathcal{D}$;
  *Model prediction for the selected example*
  $\hat{y}^{(i)} \leftarrow \hat{f}_{\theta^{(n)}}(x^{(i)})$;
  *Per-example loss between the prediction and the ground truth*
  $\mathcal{L}^{(i)}\left(\theta^{(n)}\right) \leftarrow \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$;
  *Gradient of the per-example loss, since it is differentiable*
  $g \leftarrow \nabla_\theta \mathcal{L}^{(i)}\left(\theta^{(n)}\right)$;
  *Error minimization via a gradient descent step*
  $\theta^{(n+1)} \leftarrow \theta^{(n)} - \eta\, g$;
  $n \leftarrow n + 1$;
**Return** $\theta^{(n)}$

Despite the lack of theoretical guarantees of convergence to a global optimum, the simple procedure of random initialization followed by iterative gradient-based updates is sufficient in practice to train complex NN.

## 1.5 Evaluation

Now that the process of training a model has been described, the question of how to evaluate it must be addressed. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset $\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model

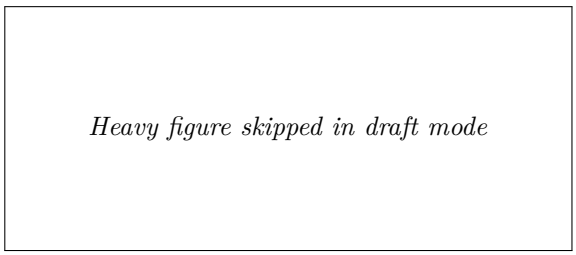


Figure 1.6: *Figure skipped in draft mode (set* `\draftfiguresfalse` *to render it).*

will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\mathcal{L}}_{\text{train}}$, is studied. This is an instance of the performance measure $P$ defined in Equation (1.3), restricted to $\mathcal{D}_{\text{train}}$:

$$\bar{\mathcal{L}}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{train}}} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.18}$$

If $\bar{\mathcal{L}}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. Its ability to generalize must now be studied. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\mathcal{L}}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x^{(i)},y^{(i)})\in\mathcal{D}_{\text{test}}} \mathcal{L}\left(\hat{f}_\theta(x^{(i)}), y^{(i)}\right). \tag{1.19}$$

If $\bar{\mathcal{L}}_{\text{test}} \gg \bar{\mathcal{L}}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to

the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\mathcal{L}}_{\text{test}} \approx \bar{\mathcal{L}}_{\text{train}}$, this indicates that the training procedure has successfully produced a NN that generalizes well beyond the training data [Kawaguchi et al., 2022]. The resulting model can therefore be considered suitable for the task for which it was trained, marking the end of the NN training process.

## 1.6 Conclusion

This chapter has introduced the fundamental concepts underlying DL, including the general objective of NN models, their structure, the training procedure, and their evaluation. For clarity of presentation, several simplifying assumptions have been made throughout this chapter. The problem has been considered in a regression setting, using a MLP trained with SGD and a batch size of one. These choices were made deliberately to expose the core mechanisms of DL as clearly as possible.

Despite these simplifications, the framework introduced here captures mechanisms that are shared across a broad range of DL methods: A sequence of parameter updates, each based on information about the loss around the current parameters. The apparent simplicity of this learning procedure therefore contrasts with the complexity of the functions that NN can ultimately represent. Understanding how such complex behavior emerges from a relatively simple optimization process remains an important challenge in the study of DL.

Mathematical tools allow us to describe the functions represented by NN and the procedures used to train them. However, they do not yet fully explain their remarkable empirical performance. Understanding the origin of this success therefore requires looking beyond the optimization procedure itself.

One common feature across NN architectures is the presence of intermediate representations. Whether considering a MLP, a CNN, an LSTM, or an architecture based on attention, the input is progressively transformed into intermediate representations before producing the final output. Their presence provides a common perspective for understanding how NNs transform information. The collection of these intermediate representations is referred to as the Latent Space (LS). Studying this LS can provide insights into how NNs learn and generalize, which motivates the following chapter.

# Acronyms

**CNN** Convolutional Neural Network 2, 15

**DL** Deep Learning 1–7, 9, 14, 15

**LLM** Large Language Model 3
**LS** Latent Space 15
**LSTM** Long Short-Term Memory 2, 15

**MLP** MultiLayer Perceptron 5–7, 15
**MSE** Mean Squared Error 4

**NN** Neural Network 1–10, 12, 13, 15

**SGD** Stochastic Gradient Descent 10, 12, 15



Compiled on 2/9/2026 at 11:58

# Bibliography


[Brown et al., 2020] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

[Covington et al., 2016] Covington, P., Adams, J., and Sargin, E. (2016). Deep neural networks for youtube recommendations. In Proceedings of the 10th ACM conference on recommender systems, pages 191–198.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). Deep learning, volume 1. MIT press Cambridge.

[Hilbert and López, 2011] Hilbert, M. and López, P. (2011). The world's technological capacity to store, communicate, and compute information. science, 332(6025):60–65.

[Hochreiter and Schmidhuber, 1997] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural computation, 9(8):1735–1780.

[Hornik et al., 1989] Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. Neural networks, 2(5):359–366.

[Jumper et al., 2021] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. (2021). Highly accurate protein structure prediction with alphafold. nature, 596(7873):583–589.

[Kawaguchi et al., 2022] Kawaguchi, K., Bengio, Y., and Kaelbling, L. (2022). Generalization in Deep Learning, page 112–148. Cambridge University Press.

[Krizhevsky et al., 2012] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

[LeCun et al., 1989] LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., and Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural computation, 1(4):541–551.

[McCulloch and Pitts, 1943] McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The bulletin of mathematical biophysics, 5(4):115–133.

[Minsky and Papert, 1969] Minsky, M. and Papert, S. (1969). An introduction to computational geometry. Cambridge tiass., HIT, 479(480):104.

[Mitchell, 1997] Mitchell, T. M. (1997). Machine Learning. McGraw-Hill, New York.

[Raina et al., 2009] Raina, R., Madhavan, A., Ng, A. Y., et al. (2009). Large-scale deep unsupervised learning using graphics processors. In Icml, volume 9, pages 873–880.

[Rosenblatt, 1958] Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. Psychological review, 65(6):386.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. nature, 323(6088):533–536.

[Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815.

[Telgarsky, 2016] Telgarsky, M. (2016). Benefits of depth in neural networks. In Conference on learning theory, pages 1517–1539. PMLR.

[Vaswani et al., 2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.